



UNIT I — INTRODUCTION TO FORMAL LANGUAGES AND FINITE AUTOMATA

Subject: Theory Of Computations

Branch: Computer Engineering | Sem: Semester 5 | SPPU 2024 Pattern

SYLLABUS TOPICS:

- Basic Concepts: Finite and infinite set, Symbols, Strings: Empty String, Substring of a string, Concatenation of strings
- Language: Formal Language Definition, Finite representation of languages
- Operations on languages: Union, Concatenation, Kleene star and Kleene plus
- Concept of Basic Machine
- FA without output: Finite State Machines (FSM), Deterministic and Nondeterministic FA (DFA & NFA), epsilon NFA
- Conversion of NFA with epsilon moves to NFA, Conversion of NFA to DFA, and Conversion of NFA with epsilon moves to DFA
- Minimization of DFAs
- FA with output: Moore and Mealy machines - Definition, Construction, Inter-Conversion

1.1 Sets, Symbols and Strings

Definition:

A set is a well defined collection of distinct objects, called elements, with no ordering or repetition implied. A symbol is an abstract, undefined entity used as a basic unit of information such as a letter or a digit. A string is a finite sequence of symbols drawn from a fixed alphabet, written one after another with no separator.

Introduction / Need:

Every topic in Theory of Computation begins with sets and strings because a formal language is nothing but a set of strings. Before we can define what a machine accepts, we must first agree on what a symbol, a string and a set actually mean in a precise mathematical sense.

SPPU asks this topic mainly through short definition and example based questions. It looks simple, but examiners often test whether students can correctly identify length, substrings and concatenation of a given string, so the notation must be learnt carefully.

1.1.1 Finite and Infinite Sets

A finite set has a countable, limited number of elements, so we can list all of them and stop. For example the set of vowels in English, {a, e, i, o, u}, has exactly 5 elements.

An infinite set has an unlimited number of elements that can never be fully listed, even though a general rule can describe them. The set of all natural numbers $N = \{0, 1, 2, 3, \dots\}$ is infinite because there is no last number.

In automata theory, an alphabet is always a finite set of symbols, but the language built from it (the set of all possible strings) is very often an infinite set. This distinction becomes important when we later study finite representation of infinite languages.

- Cardinality: the number of elements in a set, written $|S|$; for infinite sets cardinality is not a finite number.
- Empty set ($\emptyset$): a special finite set with zero elements, cardinality 0.
- Countably infinite set: an infinite set whose elements can be listed in a sequence, such as $\mathbb{N}$.
- Power set: the set of all subsets of a set S , written 2^S , always finite if S is finite.

Sr. No.	Property	Finite Set	Infinite Set
1	Number of elements	Countable and limited	Unlimited, cannot be listed fully
2	Example	$\{a, b, c\}$	Set of natural numbers $\mathbb{N}$
3	Cardinality	A fixed whole number	Not expressible as a fixed number
4	Use in automata theory	Alphabet, set of states Q	Language L , set of all strings Σ^*

1.1.2 Symbols and Alphabet

A symbol (also called a character) is an atomic, undefined unit such as 0, 1, a or b. Symbols themselves are not broken down further inside the theory.

An alphabet, written with the Greek letter Σ , is a finite, non-empty set of symbols. For example $\Sigma = \{0, 1\}$ is the binary alphabet, and $\Sigma = \{a, b, c, \dots, z\}$ is the alphabet of lower case English letters.

The choice of alphabet decides what strings and languages can even be written. A different alphabet gives a completely different set of possible strings, even for the same machine design.

- Symbols are treated as indivisible units, never split further.
- An alphabet must always be finite; an infinite alphabet is not allowed in standard automata theory.
- The same symbol can appear multiple times inside one string.
- Common alphabets used in examples: $\Sigma = \{0, 1\}$, $\Sigma = \{a, b\}$, $\Sigma = \{a, b, c\}$.

1.1.3 Strings, Empty String and Length

A string (also called a word) is a finite sequence of symbols taken from an alphabet Σ , written next to each other, such as abba over $\Sigma = \{a, b\}$.

The empty string, written as ϵ (epsilon) or sometimes λ , is the unique string that contains zero symbols. It has length 0 but it is still a valid string over every alphabet.

The length of a string w , written $|w|$, is the total count of symbols in it, counting repetitions. For example $|abba| = 4$ and $|\epsilon| = 0$.

- Σ^* denotes the set of all possible strings over Σ , including ϵ .
- Σ^+ denotes the set of all non-empty strings over Σ , so $\Sigma^+ = \Sigma^* - \{\epsilon\}$.
- Reverse of a string w , written w^R , is the string written backwards; reverse of abc is cba.
- Two strings are equal only if they have the same length and the same symbol at every position.

Sr. No.	Term	Symbol	Meaning
1	Alphabet	Σ	Finite set of symbols
2	Empty string	ϵ	String with zero symbols, length 0
3	All strings	Σ^*	$\Sigma^+ \cup \{\epsilon\}$, includes empty string
4	Non-empty strings	Σ^+	All strings except ϵ
5	Length of string	$ w $	Count of symbols in w

1.1.4 Substring, Prefix and Suffix

A substring of a string w is any contiguous block of symbols taken from w , including w itself and ϵ . For $w = abc$, the substrings are ϵ , a , b , c , ab , bc and abc .

A prefix is a substring that starts from the first symbol of w , such as a and ab for $w = abc$. A suffix is a substring that ends at the last symbol of w , such as bc and c for $w = abc$.

These ideas matter directly in automata design, because many DFA and NFA problems ask us to accept strings that start with, end with, or contain a certain substring.

- A proper prefix or proper suffix excludes the string w itself.
- Every string is trivially a substring, a prefix and a suffix of itself.
- The number of substrings of a string of length n (counting ϵ) can be at most $n(n+1)/2 + 1$ distinct ones.

1.1.5 Concatenation of Strings

Concatenation joins two strings x and y into a single string, written xy , by placing y immediately after x with no gap. If $x = ab$ and $y = ba$, then $xy = abba$.

Concatenation with the empty string never changes a string: $x\epsilon = \epsilon x = x$ for every string x . This makes ϵ act like the identity element for concatenation, similar to zero in addition.

Concatenation is associative, meaning $(xy)z = x(yz)$, but it is not commutative in general, because xy is usually not equal to yx .

- Length rule: $|xy| = |x| + |y|$.
- Repetition notation: w^n means w concatenated with itself n times; $w^0 = \epsilon$.
- Concatenation is the basic building block used later to define language concatenation.

Worked Example 1

Given: Let $\Sigma = \{a, b\}$ and $w = abba$.

Find: The length of w , all prefixes of w , and the reverse of w .

Solution:

Step 1: Write down the string: $w = abba$.

Step 2: Count the symbols one by one: a, b, b, a giving 4 symbols.

Step 3: List every prefix from length 0 to 4: $\epsilon, a, ab, abb, abba$.

Step 4: Reverse the string by writing symbols from last to first: a, b, b, a reversed gives $abba$.

Ans: $|w| = 4$, Prefixes = $\{\epsilon, a, ab, abb, abba\}$, $w^R = abba$

Applications:

- Lexical analysis phase of a compiler, where source code is read as a string over the ASCII alphabet.
- Pattern matching tools such as grep, which search for a substring inside a large text file.
- DNA sequence analysis in bioinformatics, where a strand is treated as a string over the alphabet {A, C, G, T}.
- Password validation systems, which check length and substring rules on an input string.

KEY POINTS — Sets, Symbols and Strings

A language is built entirely from strings, and a string is built entirely from symbols of a fixed alphabet — this chain is the foundation of the whole subject.

Remember $\Sigma^* = \Sigma^+ \cup \{\epsilon\}$; students often forget that Σ^* includes the empty string while Σ^+ does not.

Common exam trap: confusing substring with subsequence — a substring must be contiguous, a subsequence need not be.

$|xy| = |x| + |y|$ is used repeatedly in later proofs about pumping lemma and DFA state counts.

SPPU frequently asks to list all prefixes, suffixes or substrings of a small given string — practice this by hand.

SPPU EXPECTED QUESTIONS — Sets, Symbols and Strings

Q1. Define set, symbol, alphabet and string with one example each. (5 marks)

Q2. Explain the difference between Σ^* and Σ^+ with examples. (5 marks)

Q3. For $w = 1011$, list all prefixes, suffixes and substrings of w . (6 marks)

Q4. Differentiate between finite set and infinite set with reference to automata theory. (6 marks)

Q5. Explain concatenation of strings with its properties and one solved example. (6 marks)

1.2 Formal Language and Operations on Languages

Definition:

A formal language L over an alphabet Σ is defined as any subset of Σ^* , the set of all possible strings over Σ . A language may be finite, infinite, or even empty, and it can be described using a rule, a grammar, or a machine.

Introduction / Need:

A programming language, an English sentence, or a set of valid email addresses can all be modelled as a formal language once we fix an alphabet and describe which strings belong to it. This is why Theory of Computation calls itself the mathematics of languages.

SPPU expects students to move comfortably between three views of a language: as a set described in words, as a set built using regular operations, and as a set accepted by a machine. This topic builds the middle view.

1.2.1 Formal Language Definition

Formally, if Σ is an alphabet, then any subset $L \subseteq \Sigma^*$ is called a language over Σ . Since Σ^* itself can be infinite, a language can contain infinitely many strings.

Two extreme languages always exist for any alphabet: the empty language $\emptyset$, which contains no strings at all, and Σ^* itself, which contains every possible string.

A language is different from a single string — a language is a collection (set) of strings, while a string is one sequence of symbols. Confusing the two is a very common beginner mistake.

- $L = \{w \in \Sigma^* \mid w \text{ has even length}\}$ is an example of a language defined by a property.
- $L = \{\epsilon\}$ is a language containing exactly one string, the empty string — this is not the same as $\emptyset$.
- A language can be finite (a fixed list of strings) or infinite (defined by a rule).

1.2.2 Finite Representation of Languages

Even though a language can have infinitely many strings, we still need a finite way to describe or represent it, otherwise no computer could ever store or check it.

Four common finite representations are used in this subject: listing (only for finite languages), set-builder notation with a property, regular expressions, and machines such as DFA, NFA or grammars.

This idea of finite representation of an infinite set is the central engineering problem solved by finite automata — a machine with a fixed, finite number of states can still accept an infinite language.

- Listing method: $L = \{a, aa, aaa\}$ works only when L is finite.
- Set-builder method: $L = \{a^n b^n \mid n \geq 0\}$ describes an infinite language in one line.
- Regular expression method: $L = (a+b)^* ab$ describes all strings ending in ab .
- Machine method: a DFA, NFA or grammar accepts or generates exactly the strings of L .

1.2.3 Union of Languages

Since a language is a set, all standard set operations apply to it directly. The union of two languages L_1 and L_2 , written $L_1 \cup L_2$, is the language containing every string that belongs to L_1 or L_2 or both.

Union is useful when a machine needs to accept two different patterns at once, for example strings that either start with a or end with b .

Union of languages is commutative and associative, exactly like union of ordinary sets, because a language is just a set of strings.

- $L_1 \cup L_2 = \{w \mid w \in L_1 \text{ or } w \in L_2\}$.
- If $L_1 = \{a, ab\}$ and $L_2 = \{b, ab\}$, then $L_1 \cup L_2 = \{a, b, ab\}$, note ab appears only once.

1.2.4 Concatenation of Languages

The concatenation of two languages L_1 and L_2 , written $L_1 L_2$ or $L_1.L_2$, is the set of all strings formed by taking one string from L_1 and joining it with one string from L_2 .

This is different from string concatenation, because here we are combining every possible pair of strings, one from each language, not just two fixed strings.

If either L_1 or L_2 is empty ($\emptyset$), the concatenation $L_1 L_2$ is also empty, because there is no string to pick from the empty language.

- $L_1 L_2 = \{xy \mid x \in L_1 \text{ and } y \in L_2\}$.

- $L\{\epsilon\} = L$ for any language L , because concatenating with the language containing only ϵ changes nothing.
- $L\emptyset = \emptyset L = \emptyset$ for any language L .

Sr. No.	L1	L2	L1L2 (Result)
1	{a, b}	{c}	{ac, bc}
2	{ab}	{cd, ef}	{abcd, abef}
3	{ ϵ }	{x, y}	{x, y}
4	$\emptyset$	{p, q}	$\emptyset$

1.2.5 Kleene Star and Kleene Plus

Kleene star, written L^* , is the language formed by concatenating zero or more strings from L in any combination, so L^* always includes ϵ even if L itself does not contain ϵ .

Kleene plus, written L^+ , is almost the same idea but requires at least one string from L , so $L^+ = LL^*$ and it does not automatically add ϵ unless L already contains ϵ .

These two operators are the reason finite automata can accept infinite languages from a finite alphabet — repeating a small finite pattern any number of times generates infinitely many strings.

- $L^* = L^0 \cup L^1 \cup L^2 \cup L^3 \cup \dots$ where $L^0 = \{\epsilon\}$.
- $L^+ = L^1 \cup L^2 \cup L^3 \cup \dots$, so $L^+ = L^* - \{\epsilon\}$ only when $\epsilon \notin L$.
- If $L = \{a\}$, then $L^* = \{\epsilon, a, aa, aaa, \dots\}$ and $L^+ = \{a, aa, aaa, \dots\}$.

Formula Name: Kleene Star of a Language

Expression: $L^* = \bigcup_{n=0}^{\infty} L^n$

Where:

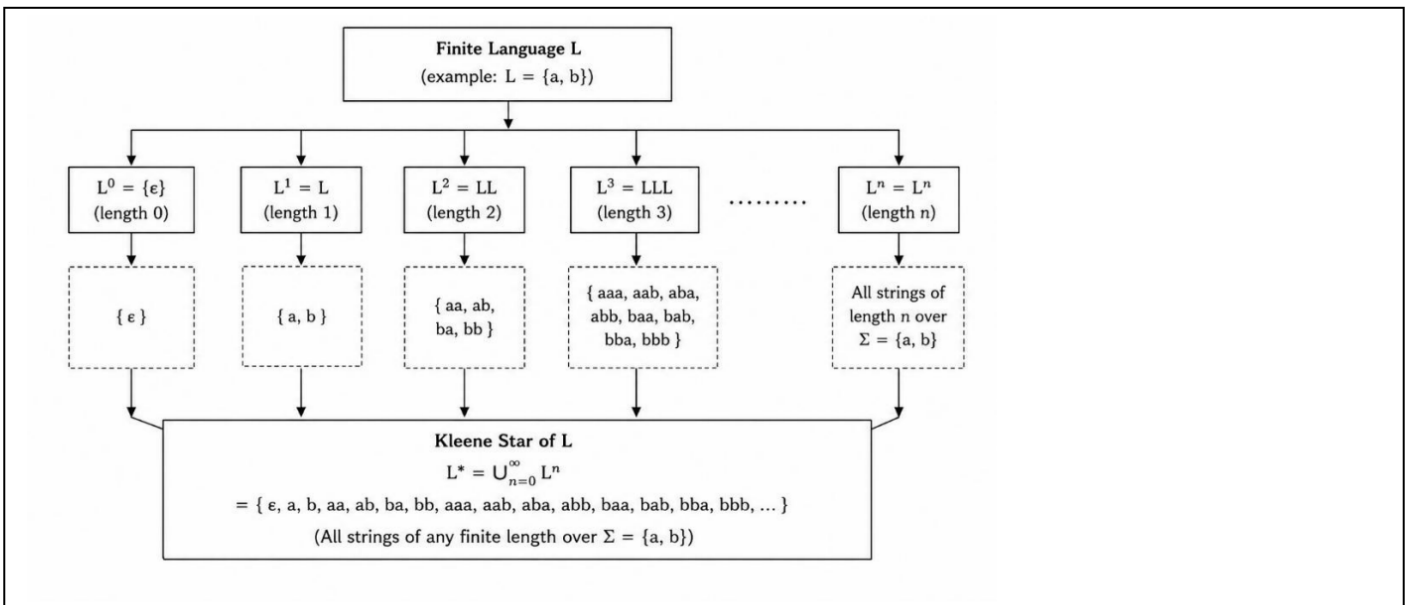
L = given language over alphabet Σ

L^n = L concatenated with itself n times

$L^0 = \{\epsilon\}$ by definition

Condition: Applies to any language L , finite or infinite; always produces an infinite language unless $L = \emptyset$ or $L = \{\epsilon\}$.

Fig 1.2.2 — Kleene star operation building an infinite language from a finite one



Worked Example 1

Given: Let $\Sigma = \{a, b\}$, $L1 = \{a, ab\}$ and $L2 = \{b, ba\}$.

Find: $L1 \cup L2$ and $L1L2$.

Solution:

Step 1: Compute union by combining all distinct strings of $L1$ and $L2$.

Step 2: $L1 \cup L2 = \{a, ab, b, ba\}$, no repeated strings here so nothing is removed.

Step 3: Compute concatenation by pairing every string of $L1$ with every string of $L2$.

Step 4: Pairs are: $a.b=ab$, $a.ba=aba$, $ab.b=abb$, $ab.ba=abba$, giving $L1L2 = \{ab, aba, abb, abba\}$.

Ans: $L1 \cup L2 = \{a, ab, b, ba\}$; $L1L2 = \{ab, aba, abb, abba\}$

Worked Example 2

Given: Let $L = \{a, b\}$ over $\Sigma = \{a, b\}$.

Find: L^* up to strings of length 2, and L^+ up to length 2.

Solution:

Step 1: Start with $L^0 = \{\epsilon\}$ which is always part of L^* .

Step 2: Compute $L^1 = L = \{a, b\}$.

Step 3: Compute $L^2 = LL = \{aa, ab, ba, bb\}$.

Step 4: Union these levels: L^* (up to length 2) = $\{\epsilon, a, b, aa, ab, ba, bb\}$, and L^+ removes only ϵ .

Ans: L^* (partial) = $\{\epsilon, a, b, aa, ab, ba, bb\}$; L^+ (partial) = $\{a, b, aa, ab, ba, bb\}$

Advantages:

- Set operations let a complex language be built from small, simple languages step by step.
- Union directly models a machine that must accept more than one pattern.
- Concatenation models sequential structure, such as an identifier followed by a number in a compiler token.
- Kleene star models unlimited repetition using only a finite alphabet and finite operations.
- These operations are closed on regular languages, which keeps later automata proofs simple.

Disadvantages:

- Union and concatenation of automata increase the number of states, which can make machines harder to draw by hand.
- Kleene star can make a language much larger than needed if the base language is not chosen carefully.
- Nested operations (star of a union of a concatenation) quickly become difficult to trace manually without a systematic method.

Applications:

- Regular expressions in text editors and search tools rely directly on union, concatenation and Kleene star.
- Compiler lexical analysers build token languages such as identifier = letter(letter+digit)* using these operations.
- Network packet filters describe valid address patterns using union of allowed prefixes.
- Password strength checkers describe valid password languages as concatenation of character-class languages.

KEY POINTS — Formal Language and Operations on Languages

A language is a SET of strings, never a single string; treat all language operations as set operations.

Kleene star always contains ϵ , even if the base language does not — this is the single most common exam trap in this topic.

$L^+ = LL^*$ is the relation SPPU most frequently asks students to prove or apply.

Concatenation of languages pairs every string of L_1 with every string of L_2 , not just corresponding positions.

If either language in a concatenation is $\emptyset$, the result is always $\emptyset$ — students often forget this special case.

SPPU EXPECTED QUESTIONS — Formal Language and Operations on Languages

Q1. Define formal language with a suitable example. (5 marks)

Q2. Explain finite representation of an infinite language with examples. (5 marks)

Q3. For $L_1 = \{a, ab\}$ and $L_2 = \{b, ba\}$, find $L_1 \cup L_2$, L_1L_2 . (6 marks)

Q4. Define Kleene star and Kleene plus and prove that $L^+ = LL^*$. (6 marks)

Q5. Explain union, concatenation, Kleene star and Kleene plus operations on languages with examples. (8 marks)

1.3 Concept of Basic Machine

Definition:

A basic machine, in automata theory, is an abstract computational model that reads an input string symbol by symbol and changes its internal condition, called a state, according to fixed rules, in order to decide whether the string is accepted.

Introduction / Need:

Before studying any specific automaton, it helps to understand what all these machines have in common. Every automaton studied in this unit is built from the same small set of ideas: states, an input, and a rule for moving between states.

This topic is intentionally kept conceptual rather than mathematical, because the exact formal definitions are given separately for DFA, NFA and the machines with output later in the unit.

1.3.1 Components of a Basic Machine

Every basic machine has an input tape or input string, a finite control that holds the current state, and a transition rule that decides the next state for each input symbol read.

The finite control can only be in one defined state at any moment, and the total number of possible states is always finite, no matter how long the input string is.

A special starting state marks where the machine begins, and one or more final (accepting) states mark where the machine may correctly stop and accept the string.

- Input tape: holds the string to be processed, read one symbol at a time, usually left to right.
- Finite control: the part of the machine holding the current state, implemented conceptually as a set of states Q .
- Transition function: the rule δ that tells the machine which state to go to next.
- Start state and final states: special members of Q that mark where processing begins and where acceptance is decided.

1.3.2 Classification of Basic Machines

Basic machines are broadly split into two families based on what they produce at the end of processing: machines without output and machines with output.

A machine without output, such as DFA or NFA, only answers yes or no — it accepts or rejects the input string based on whether it ends in a final state.

A machine with output, such as Moore or Mealy machine, produces an actual output symbol at each step or state, in addition to just accepting or rejecting.

- FA without output: DFA, NFA, epsilon-NFA — studied for language acceptance.
- FA with output: Moore machine and Mealy machine — studied for output generation, such as a sequence detector.

Sr. No.	Machine Type	Produces Output?	Example Use
1	DFA / NFA	No, only accept or reject	Checking if a string matches a pattern
2	Moore Machine	Yes, output tied to state	Traffic light controller
3	Mealy Machine	Yes, output tied to transition	Sequence detector circuit

Applications:

- Vending machine controllers, which move through states such as coin-inserted and dispense based on basic machine logic.
- Traffic signal controllers, which cycle through a fixed finite number of states.
- Lexical analysers in compilers, which are literally implemented as finite automata.

KEY POINTS — Concept of Basic Machine

Every automaton in this unit shares three parts: states, an alphabet of input symbols, and a transition rule — learn this once and it applies everywhere.

The key split to remember is FA without output (DFA, NFA) versus FA with output (Moore, Mealy).

Common exam trap: students confuse the finite control (current state) with the finite automaton itself; the control is only one part of the machine.

SPPU EXPECTED QUESTIONS — Concept of Basic Machine

- Q1. Explain the basic components of a finite automaton with a block diagram. (5 marks)
- Q2. Differentiate between finite automata with output and without output. (5 marks)

1.4 Finite State Machine and Deterministic Finite Automata (DFA)

Definition:

A Deterministic Finite Automaton (DFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where Q is a finite set of states, Σ is a finite input alphabet, $\delta: Q \times \Sigma \rightarrow Q$ is the transition function, $q_0 \in Q$ is the start state, and $F \subseteq Q$ is the set of final states, such that for every state and every input symbol there is exactly one defined next state.

Introduction / Need:

A finite state machine (FSM) is the general name for any machine with a finite number of states, and DFA is its simplest, most disciplined form. In a DFA, the next move is never in doubt — one state, one symbol, one destination, always.

SPPU treats DFA as the base case for everything that follows in this unit: NFA, epsilon-NFA and minimization are all explained by comparing them back to this deterministic model, so this topic must be mastered thoroughly.

1.4.1 Formal Definition and 5-Tuple Notation

The 5-tuple $(Q, \Sigma, \delta, q_0, F)$ captures everything about a DFA in compact mathematical notation, and every DFA design problem in the exam expects the answer to state this tuple explicitly.

The word deterministic means that $\delta(q, a)$ gives exactly one next state for every state q and every symbol a in Σ — there is never a choice and never a missing transition.

Because δ is a total function (defined for every state-symbol pair), a DFA is always in exactly one state after reading any prefix of the input, which makes it simple to simulate on a computer.

- Q : finite, non-empty set of states, e.g. $Q = \{q_0, q_1, q_2\}$.

- Σ : finite input alphabet, e.g. $\Sigma = \{0, 1\}$.
- $\delta: Q \times \Sigma \rightarrow Q$, the transition function, total and single-valued.
- $q_0 \in Q$: the unique start state where processing always begins.
- $F \subseteq Q$: the set of accepting or final states.

Formula Name: DFA Formal Definition
Expression: $M = (Q, \Sigma, \delta, q_0, F)$
Where:
 Q = finite set of states

 Σ = finite input alphabet

 $\delta = Q \times \Sigma \rightarrow Q$, transition function

 q_0 = initial state, $q_0 \in Q$
 F = set of final states, $F \subseteq Q$

Condition: δ must be defined for every (state, symbol) pair — this is what makes the automaton deterministic and total.

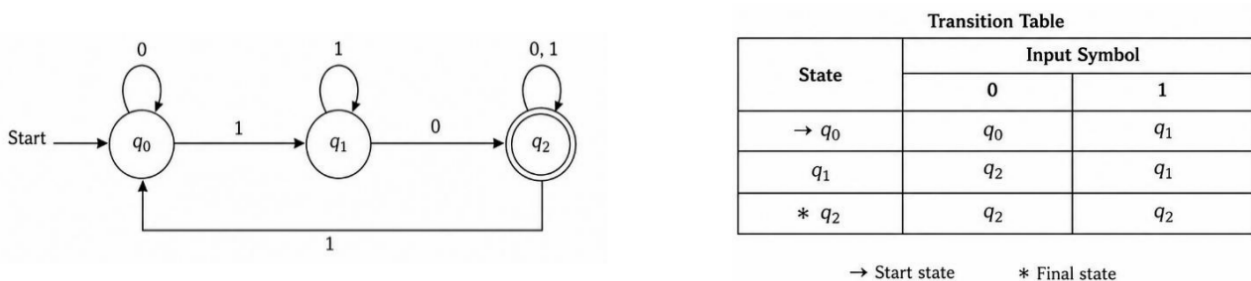
1.4.2 Transition Diagram and Transition Table

A transition diagram is a directed graph where each state is a circle, each final state is a double circle, and each transition is a labelled arrow showing which symbol causes that move.

A transition table is the same information written as a grid, with states as rows, input symbols as columns, and the next state written in each cell — this form is easier to use for hand simulation.

Both representations must always agree exactly; SPPU frequently asks to convert one form into the other during the exam.

- Start state is marked with an incoming arrow that has no source state.
- Final states are drawn as double circles (or marked with an asterisk in tables).
- Every row of the transition table must have one entry per input symbol, with no blanks, since δ is total.

Fig 1.4.1 — Transition diagram of a sample DFA


1.4.3 Language Accepted by a DFA

A string w is accepted by a DFA if, starting from q_0 and following the transitions for each symbol of w in order, the machine ends up in a state that belongs to F .

The extended transition function δ^* generalises δ from single symbols to entire strings, so $\delta^*(q_0, w)$ gives the final state reached after processing the whole string w .

The language of the DFA, written $L(M)$, is the set of all strings that lead from q_0 to some state in F : $L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \in F \}$.

- Base case: $\delta^*(q, \epsilon) = q$ for any state q (reading nothing leaves the state unchanged).
- Recursive case: $\delta^*(q, wa) = \delta(\delta^*(q, w), a)$ for a string w followed by symbol a .
- A DFA rejects any string that ends in a non-final state, even a dead/trap state.

1.4.4 Design Examples of DFA

Designing a DFA starts by identifying what memory the machine must track using its states — for example, whether the count of a's seen so far is even or odd.

A common design pattern uses a small number of states, each representing one meaningful condition, plus a trap (dead) state to absorb any string that has already failed the pattern.

Practice with small alphabets like $\{0,1\}$ or $\{a,b\}$ builds the intuition needed for more complex machines used in NFA and Moore/Mealy design later in the unit.

- Design DFA to accept strings ending in a specific substring, e.g. ending in 'ab'.
- Design DFA to accept strings with an even number of a particular symbol.
- Design DFA to accept strings of length divisible by a fixed number.
- Design DFA to accept strings containing a fixed substring anywhere.

1.4.5 Simulation of a DFA (Algorithm)

Simulating a DFA on a computer is straightforward because the transition function is single-valued — the algorithm simply looks up one entry in the transition table at each step.

This is why DFA based pattern matching, such as used in lexical analysers, is extremely fast: each input symbol is processed in constant time.

The algorithm below shows the general simulation logic used to check whether a DFA accepts a given string.

```
function DFA_ACCEPTS(M, w):  
    state = M.q0  
    for each symbol a in w:  
        state = M.delta(state, a)  
    if state in M.F:  
        return ACCEPT  
    else:  
        return REJECT
```

Time Complexity: $O(n)$, where n is the length of input string w **Space Complexity:** $O(1)$ extra space (only current state is stored)

Worked Example 1

Given: Design a DFA over $\Sigma = \{a, b\}$ that accepts all strings ending in 'ab'.

Find: The states, transition table and diagram logic for this DFA.

Solution:

Step 1: Identify what must be remembered: how much of the pattern 'ab' has been matched so far.

Step 2: Define states: q_0 = no match yet (start), q_1 = last symbol read was 'a', q_2 = pattern 'ab' just completed (final state).

Step 3: Build transitions: from q_0 on a go to q_1 , on b stay at q_0 ; from q_1 on a stay at q_1 , on b go to q_2 ; from q_2 on a go to q_1 , on b go to q_0 .

Step 4: Verify with sample string 'aab': $q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$, which is a final state, so 'aab' is accepted.

Ans: DFA = ($Q=\{q_0, q_1, q_2\}$, $\Sigma=\{a, b\}$, δ as above, q_0 , $F=\{q_2\}$); string 'aab' is ACCEPTED

Worked Example 2

Given: Design a DFA over $\Sigma = \{a, b\}$ that accepts strings with an even number of a's.

Find: Number of states needed, and whether the string 'baba' is accepted.

Solution:

Step 1: Identify memory needed: only whether the count of a's seen so far is even or odd, so 2 states suffice.

Step 2: Define states: q_{Even} = even number of a's so far (start and final), q_{Odd} = odd number of a's so far.

Step 3: Transitions: on b, stay in the same state (b does not change the a-count); on a, switch states (q_{Even} to q_{Odd} and q_{Odd} to q_{Even}).

Step 4: Trace 'baba': $q_{\text{Even}} \xrightarrow{b} q_{\text{Even}} \xrightarrow{a} q_{\text{Odd}} \xrightarrow{b} q_{\text{Odd}} \xrightarrow{a} q_{\text{Even}}$, which is the final state.

Ans: 2 states required; string 'baba' has 2 a's (even) and is ACCEPTED, ending in q_{Even}

Advantages:

- Simulation runs in linear time $O(n)$ with only one current state stored, making DFA extremely memory efficient.
- Behaviour is fully predictable since each state-symbol pair has exactly one next state.
- Directly implementable in hardware or software as a simple lookup table.
- Easy to prove correctness because there is no branching or guessing involved.
- Forms the basis of fast pattern matching used in real compilers and text search tools.

Disadvantages:

- Can require many more states than an equivalent NFA for the same language, since there is no non-determinism to share work between paths.
- Manually designing a DFA for a complex pattern can be error-prone due to the need for a transition on every symbol from every state.
- Cannot directly represent languages that are not regular, such as balanced parentheses of unbounded depth.

Applications:

- Lexical analysers in compilers such as GCC and Clang, which tokenise source code using DFA based pattern matching.
- Text search utilities such as grep, which compile regular expressions into DFAs internally for speed.
- Network intrusion detection systems, which scan packet payloads for known attack signatures using DFA matching.

- Digital vending machines and elevator controllers, modelled directly as DFAs with a small fixed set of states.
- Protocol validators that check whether a sequence of network messages follows an allowed order.

KEY POINTS — Finite State Machine and Deterministic Finite Automata (DFA)

Memorise the 5-tuple $(Q, \Sigma, \delta, q_0, F)$ — SPPU always expects this notation to be written explicitly in any DFA answer.

δ in a DFA is total and single-valued: exactly one transition per state-symbol pair, never zero, never more than one.

Common exam trap: forgetting the trap/dead state when a transition genuinely leads nowhere useful — δ must still be defined there.

$L(M) = \{ w \mid \delta^*(q_0, w) \in F \}$ is the standard definition examiners expect for 'language accepted by a DFA'.

DFA simulation is $O(n)$ time and $O(1)$ extra space — this efficiency is the main reason DFA is preferred over NFA in real systems.

SPPU EXPECTED QUESTIONS — Finite State Machine and Deterministic Finite Automata (DFA)

Q1. Define DFA formally using 5-tuple notation with explanation of each component. (5 marks)

Q2. Design a DFA over $\Sigma = \{a,b\}$ that accepts strings ending in 'ab'. Draw the transition diagram. (6 marks)

Q3. Design a DFA that accepts strings over $\{a,b\}$ having an even number of a's, with transition table. (8 marks)

Q4. Explain the extended transition function δ^* with its recursive definition. (6 marks)

Q5. Design a DFA to accept all strings over $\{0,1\}$ whose length is a multiple of 3, with justification. (8 marks)

1.5 Nondeterministic Finite Automata (NFA)

Definition:

A Nondeterministic Finite Automaton (NFA) is a 5-tuple $(Q, \Sigma, \delta, q_0, F)$ where $\delta: Q \times \Sigma \rightarrow 2^Q$ is the transition function, allowing zero, one, or more next states for a given state and input symbol, and a string is accepted if at least one such path ends in a final state.

Introduction / Need:

In a DFA the next move is always fixed, but in an NFA the machine is allowed to have several possible next states, or even none at all, for the same state and symbol. This freedom makes NFAs easier to design by hand, even though a real computer cannot literally guess.

SPPU uses NFA as the bridge topic between simple DFA design and the more complex conversion algorithms (NFA to DFA, epsilon elimination) that dominate the rest of this unit, so understanding NFA acceptance clearly here saves time later.

1.5.1 Formal Definition of NFA

An NFA has the same five components as a DFA, but its transition function $\delta: Q \times \Sigma \rightarrow 2^Q$ maps each state-symbol pair to a set of possible next states, rather than exactly one.

2^Q , the power set of Q , represents all possible subsets of states, since $\delta(q, a)$ could be the empty set (no move possible), a single state, or several states at once.

A string is accepted by the NFA if there exists at least one sequence of choices that leads from q_0 to some state in F — it does not matter if other choices lead to rejection.

- Q, Σ, q_0, F have exactly the same meaning as in a DFA.
- $\delta: Q \times \Sigma \rightarrow 2^Q$ allows multiple or zero next states, unlike the DFA's single next state.
- Acceptance is existential: 'there exists a path' to a final state, not 'every path' must reach one.

Formula Name: NFA Formal Definition

Expression: $M = (Q, \Sigma, \delta, q_0, F)$, $\delta: Q \times \Sigma \rightarrow 2^Q$

Where:

Q = finite set of states

Σ = finite input alphabet

δ = transition function returning a SET of states

q_0 = initial state

F = set of final states

Condition: Accepts w if at least one computation path on w ends in a state belonging to F .

1.5.2 Difference between DFA and NFA

The core difference is in δ : a DFA gives exactly one next state, while an NFA can give many, none, or exactly one — DFA is technically a special, restricted case of NFA.

NFAs are often much easier and more compact to design by hand for a given pattern, because a state does not need to fully commit to one interpretation immediately.

Despite the apparent extra power, NFA and DFA are proven to be equally powerful — every NFA has an equivalent DFA, which is exactly what the subset construction algorithm in the next topic produces.

- Number of states needed for the same language can be exponentially fewer in an NFA compared to a DFA in the worst case.
- NFA execution can conceptually be imagined as many parallel copies of the machine running at once, one per possible current state.
- Both DFA and NFA recognise exactly the class of regular languages — neither is more powerful in terms of what languages they can express.

Sr. No.	Feature	DFA	NFA
1	Transition function	$\delta: Q \times \Sigma \rightarrow Q$ (single state)	$\delta: Q \times \Sigma \rightarrow 2^Q$ (set of states)
2	Number of moves per symbol	Exactly one	Zero, one, or many
3	Ease of hand design	Harder, needs full determinism	Easier, allows guessing
4	Execution model	Single current state	Set of possible current states

Sr. No.	Feature	DFA	NFA
5	Expressive power	Regular languages	Regular languages (same as DFA)

1.5.3 Design Examples of NFA

NFA design usually starts from the pattern description directly, letting a new branch of states handle each interpretation without merging them upfront.

A very common pattern is designing an NFA to accept strings containing a certain substring anywhere, since the machine can simply 'guess' where the substring begins.

Because the exam commonly asks to compare NFA and its equivalent DFA, it is good practice to sketch both for the same small language while learning.

- NFA accepting strings over $\{a,b\}$ that contain 'ab' as a substring, using non-determinism to guess the start of the match.
- NFA accepting strings ending in a fixed suffix, using branching states.
- NFA where the same symbol from one state leads to two different states, showing true non-determinism.

1.5.4 Language Acceptance in NFA

The extended transition function for NFA, δ^* , is defined over sets of states, since after each symbol the machine can be thought of as being in a set of possible current states simultaneously.

A string w is accepted if the set $\delta^*(q_0, w)$ has at least one element in common with F , that is, $\delta^*(q_0, w) \cap F \neq \emptyset$.

This 'at least one accepting path' rule is exactly what the subset construction algorithm (studied in the next topic) converts into a fully deterministic equivalent machine.

- $\delta^*(q, \epsilon) = \{q\}$.
- $\delta^*(q, wa) = \bigcup \text{over all states } p \text{ in } \delta^*(q, w) \text{ of } \delta(p, a)$.
- $L(M) = \{ w \in \Sigma^* \mid \delta^*(q_0, w) \cap F \neq \emptyset \}$.

Worked Example 1

Given: Design an NFA over $\Sigma = \{a, b\}$ that accepts strings containing 'ab' as a substring.

Find: The states, transitions and whether 'baab' is accepted.

Solution:

Step 1: Define states: q_0 = start, still searching for 'ab' (self loop on a and b), q_1 = just read 'a' (possible start of match), q_2 = matched 'ab' (final state, self loop on a and b to keep accepting).

Step 2: Transitions: from q_0 on a go to $\{q_0, q_1\}$ (non-deterministic branch), from q_0 on b stay q_0 ; from q_1 on b go to q_2 , from q_1 on a go to q_1 ; from q_2 on a or b stay at q_2 .

Step 3: Trace string 'baab' choosing the accepting path: $q_0 \xrightarrow{b} q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_1 \xrightarrow{b} q_2$.

Step 4: Since q_2 is a final state and was reached by at least one valid path, the NFA accepts this string.

Ans: 'baab' is ACCEPTED by the NFA since one path reaches final state q_2

Advantages:

- Much smaller and more intuitive to design by hand for complex patterns compared to an equivalent DFA.
- Naturally expresses 'guessing' constructions, useful when converting regular expressions to automata.
- Reduces design errors because branches for different cases can be added independently.
- Serves as a convenient intermediate step before building a minimal DFA.

Disadvantages:

- Cannot be executed directly on real hardware without simulating all possible states at once or converting to a DFA first.
- Simulating an NFA directly can take more time per symbol than a DFA, since multiple current states must be tracked.
- Understanding and tracing acceptance by hand is harder than DFA because multiple parallel paths must be considered.

Applications:

- Regular expression engines internally build an NFA first (via Thompson's construction) before optimising it.
- Design stage of digital circuit sequence detectors, later converted to a deterministic equivalent for implementation.
- Search algorithms that need to consider multiple possible matches starting at different positions in a string.
- Teaching and textbook examples, since NFA constructions are usually simpler to explain and draw.

KEY POINTS — Nondeterministic Finite Automata (NFA)

NFA acceptance is existential — 'at least one path reaches a final state' is enough; not every path needs to succeed.

$\delta: Q \times \Sigma \rightarrow 2^Q$ is the one formula difference between NFA and DFA definitions; write this explicitly in the exam.

Common exam trap: students think NFA is more powerful than DFA — remind yourself both accept exactly the class of regular languages.

NFA can have zero transitions for some input, meaning that branch of computation simply dies without rejecting the whole machine.

Every NFA design question is very often followed by 'convert this NFA to an equivalent DFA' — practise both together.

SPPU EXPECTED QUESTIONS — Nondeterministic Finite Automata (NFA)

Q1. Define NFA formally and explain how its transition function differs from that of a DFA. (5 marks)

Q2. Design an NFA over $\{a,b\}$ accepting strings containing 'ab' as a substring, with transition diagram. (6 marks)

Q3. Differentiate between DFA and NFA with respect to definition, execution and design. (6 marks)

Q4. Prove that NFA and DFA are equivalent in expressive power (concept level explanation). (8 marks)

Q5. Design an NFA over $\{0,1\}$ accepting strings that end with '1' preceded by at least one '0'. (8 marks)

1.6 Epsilon-NFA and Conversions (NFA- ϵ to NFA, NFA to DFA, NFA- ϵ to DFA)

Definition:

An epsilon-NFA (NFA- ϵ) is a Nondeterministic Finite Automaton extended with epsilon transitions, in which the machine can move from one state to another without consuming any input symbol, in addition to the normal symbol based transitions of an NFA.

Introduction / Need:

Epsilon transitions make automata design even more flexible, since it lets us connect smaller machines together (for example, when building an automaton for a union or concatenation of languages) without forcing an extra input symbol to be read.

However, no real hardware or program can literally move for free, so every epsilon-NFA must eventually be converted into an NFA or DFA that only moves on real input symbols — this conversion is the main algorithmic skill tested in this topic.

1.6.1 Epsilon-NFA and Epsilon Closure

In an epsilon-NFA, the transition function is extended to $\delta: Q \times (\Sigma \cup \{\epsilon\}) \rightarrow 2^Q$, meaning transitions can be labelled with ϵ as well as normal alphabet symbols.

The epsilon closure of a state q , written $\text{ECLOSE}(q)$ or $\epsilon\text{-closure}(q)$, is the set of all states reachable from q using only ϵ transitions, including q itself.

Epsilon closure is the key tool used in every conversion algorithm in this topic, because it tells us which states are 'silently' reachable before or after reading a real symbol.

- $\epsilon\text{-closure}(q)$ always includes q itself, even if q has no outgoing ϵ transitions.
- $\epsilon\text{-closure}$ is computed using a simple reachability search (BFS or DFS) following only ϵ labelled edges.
- $\epsilon\text{-closure}$ of a set of states S is the union of $\epsilon\text{-closure}$ of each individual state in S .

1.6.2 Conversion of NFA- ϵ to NFA (Elimination of Epsilon Moves)

To remove epsilon moves, we first compute the epsilon closure of every state, then redefine the transition function so that reading a real symbol a from state q jumps directly to the epsilon closure of every state reachable by a from $\epsilon\text{-closure}(q)$.

The final states of the new NFA must also be updated: any state whose epsilon closure contains an original final state must now be marked final as well, since it could silently reach acceptance.

After this construction, the resulting NFA has no epsilon transitions at all, and it accepts exactly the same language as the original epsilon-NFA.

- New transition: $\delta'(q, a) = \epsilon\text{-closure}(\bigcup \text{over } p \text{ in } \epsilon\text{-closure}(q) \text{ of } \delta(p, a))$.
- New final states: $F' = F \cup \{q \mid \epsilon\text{-closure}(q) \cap F \neq \emptyset\}$.
- Start state remains the same as in the original epsilon-NFA.

```

function REMOVE_EPSILON(M):
  for each state q in Q:
    compute ECLOSE(q)
  for each state q, each symbol a in Sigma:
    delta_new(q, a) = ECLOSE( union of delta(p, a)
                               for all p in ECLOSE(q) )
  F_new = { q | ECLOSE(q) intersects F }
  return NFA(Q, Sigma, delta_new, q0, F_new)

```

Time Complexity: $O(|Q|^2 \times |\Sigma|)$ for computing all closures and new transitions **Space Complexity:** $O(|Q|^2)$ to store epsilon closure sets for all states

1.6.3 Conversion of NFA to DFA (Subset Construction)

The subset construction algorithm converts an NFA into an equivalent DFA by treating each reachable SET of NFA states as a single DFA state — this is why the algorithm is also called the powerset construction.

Starting from $\{q_0\}$, the algorithm repeatedly computes, for every existing DFA state (a set of NFA states) and every input symbol, the union of NFA transitions from all states in that set, creating new DFA states as needed.

A DFA state is marked final if the corresponding set of NFA states contains at least one original NFA final state, matching the existential acceptance rule of NFA.

- New DFA states are subsets of the original NFA states, hence the DFA can have up to $2^{|Q|}$ states in the worst case.
- In practice only reachable subsets are constructed, so the DFA is usually much smaller than $2^{|Q|}$.
- The new start state is $\{q_0\}$; new final states are all subsets containing at least one NFA final state.

Sr. No.	Step	Action	Result
1	Initialize	Start set = ϵ -closure($\{q_0\}$)	First DFA state
2	Process each symbol	Union of NFA moves from all states in set	Candidate new DFA state
3	Check if new	Compare with existing DFA states	Add if not already present
4	Mark final states	Check intersection with NFA's F	DFA final states identified
5	Repeat	Until no unmarked states remain	Complete DFA transition table

```

function NFA_TO_DFA(N):
  start = {N.q0}
  Q_dfa = { start }, unmarked = [start]
  while unmarked is not empty:
    T = unmarked.pop()
    for each symbol a in Sigma:
      U = union of N.delta(p, a) for all p in T
      if U not in Q_dfa:
        add U to Q_dfa, unmarked.push(U)
      delta_dfa(T, a) = U
  F_dfa = { T in Q_dfa | T intersects N.F }
  return DFA(Q_dfa, Sigma, delta_dfa, start, F_dfa)

```

Time Complexity: $O(2^{|Q|} \times |\Sigma|)$ worst case, usually much smaller in practice

Space Complexity: $O(2^{|Q|})$ worst case to store all subset states

1.6.4 Conversion of NFA- ϵ to DFA (Combined Procedure)

When the original machine has epsilon moves, the subset construction is adapted by using epsilon closures at every stage instead of plain state sets, combining both previous algorithms into one pass.

The starting DFA state becomes ϵ -closure(q_0), and for every DFA state T and symbol a , the next DFA state is ϵ -closure of the union of $\delta(p, a)$ for all p in T .

This combined method is the most commonly asked numerical in this unit, because it tests epsilon closure computation and subset construction together in one worked problem.

- Start state of new DFA: ϵ -closure(q_0).
- Transition: $\delta'(T, a) = \epsilon$ -closure($\bigcup_{p \in T} \delta(p, a)$).
- Final states: any DFA state (subset) that contains at least one original final state.

Worked Example 1

Given: An NFA has states $\{q_0, q_1, q_2\}$, $\Sigma = \{a, b\}$, q_0 start, $F = \{q_2\}$, with $\delta(q_0, a) = \{q_0, q_1\}$, $\delta(q_0, b) = \{q_0\}$, $\delta(q_1, b) = \{q_2\}$, $\delta(q_2, -) = \text{no moves}$.

Find: The equivalent DFA using subset construction.

Solution:

Step 1: Start DFA state $A = \{q_0\}$.

Step 2: From A on a : union of $\delta(q_0, a) = \{q_0, q_1\}$, call this state B ; from A on b : $\delta(q_0, b) = \{q_0\} = A$ itself.

Step 3: From $B = \{q_0, q_1\}$ on a : $\delta(q_0, a) \cup \delta(q_1, a) = \{q_0, q_1\} \cup \emptyset = \{q_0, q_1\} = B$; from B on b : $\delta(q_0, b) \cup \delta(q_1, b) = \{q_0\} \cup \{q_2\} = \{q_0, q_2\}$, call this state C .

Step 4: From $C = \{q_0, q_2\}$ on a : $\delta(q_0, a) \cup \delta(q_2, a) = \{q_0, q_1\} \cup \emptyset = B$; from C on b : $\delta(q_0, b) \cup \delta(q_2, b) = \{q_0\} \cup \emptyset = A$. Mark C as final since it contains q_2 .

Ans: DFA states = $\{A, B, C\}$, start = A , final = $\{C\}$, matching original NFA language exactly

Advantages:

- Guarantees that every NFA and epsilon-NFA has an equivalent DFA, proving all three models accept the same language class.
- Subset construction is systematic and can be fully automated in software such as regular expression compilers.
- Epsilon closure simplifies combining smaller automata (union, concatenation) into one larger machine before determinizing.
- Removes the ambiguity of non-determinism, producing a machine that is directly implementable in hardware or code.

Disadvantages:

- Worst case DFA can have $2^{|Q|}$ states, causing state explosion for large NFAs.
- Computing epsilon closures for every state adds extra pre-processing time before the main construction begins.

- Manually tracing subset construction for even a medium sized NFA is tedious and error-prone without a systematic table.

Applications:

- Regular expression compilers (used inside grep, lex, and programming language regex engines) build an NFA first and then determinize it for fast matching.
- Compiler construction tools such as Lex and Flex use these exact conversions to generate lexical analysers.
- Protocol and pattern verification tools convert specification automata with epsilon moves into deterministic checkers.
- Digital circuit design tools simplify sequence detector specifications using these same closure and subset ideas.

KEY POINTS — Epsilon-NFA and Conversions (NFA- ϵ to NFA, NFA to DFA, NFA- ϵ to DFA)

ϵ -closure(q) always includes q itself — this is the single most common mistake students make while solving numericals.

Subset construction start state is ϵ -closure(q_0), not just $\{q_0\}$, whenever epsilon moves are present.

A DFA state (a subset of NFA states) is final if it contains ANY NFA final state — this is the existential rule carried over from NFA acceptance.

Worst case DFA size is $2^{|Q|}$, but only reachable subsets are actually built, so real answers are usually much smaller.

SPPU numericals almost always combine NFA- ϵ to DFA in one question — practise the combined algorithm, not just the two halves separately.

SPPU EXPECTED QUESTIONS — Epsilon-NFA and Conversions (NFA- ϵ to NFA, NFA to DFA, NFA- ϵ to DFA)

- Q1. Define epsilon-NFA and explain the concept of epsilon closure with an example. (5 marks)
- Q2. Explain the algorithm to convert NFA with epsilon moves to NFA without epsilon moves. (6 marks)
- Q3. Convert the given NFA to its equivalent DFA using subset construction, showing all steps. (8 marks)
- Q4. Explain subset construction algorithm with its time and space complexity. (6 marks)
- Q5. Convert the given epsilon-NFA directly to a DFA, showing epsilon closures and final transition table. (8 marks)

1.7 Minimization of DFAs

Definition:

Minimization of a DFA is the process of finding an equivalent DFA with the smallest possible number of states, such that the minimized machine accepts exactly the same language as the original, by merging states that are proven to be equivalent to each other.

Introduction / Need:

A DFA built directly from a design problem or from subset construction often contains extra, redundant states that behave identically to some other state. Minimization removes this redundancy without changing what the machine accepts.

This topic is important both for exams and for real systems, since a smaller DFA means less memory and faster execution when it is implemented in a compiler or a hardware sequence detector.

1.7.1 Need for Minimization

Two different states in a DFA are considered redundant if, no matter what string is fed in from that point onward, both states always agree on acceptance or rejection.

Also, states that can never be reached from the start state (unreachable states) add no value and can simply be deleted before minimization begins.

Minimizing a DFA gives the unique smallest deterministic machine for a given regular language, which is useful for comparing whether two DFAs describe the same language.

- Removing unreachable states is always the first cleanup step before applying equivalence based minimization.
- The minimized DFA is unique up to renaming of states, for any given regular language.
- Minimization never changes the language accepted — it only removes duplicate behaviour.

1.7.2 Equivalence of States

Two states p and q are said to be equivalent if, for every possible string w , $\delta^*(p, w)$ is a final state exactly when $\delta^*(q, w)$ is also a final state.

This is closely tied to the Myhill-Nerode theorem, which states that a language is regular if and only if it induces only finitely many equivalence classes over Σ^* , and the minimal DFA has exactly one state per class.

In practice, we do not check every possible string directly; instead we use the table filling method, which finds equivalent state pairs by systematic elimination.

- A final state and a non-final state are never equivalent to each other — this is the base case of distinguishability.
- Two states are distinguishable if some symbol leads them to states that are already known to be distinguishable.
- States that are never proven distinguishable after all rounds are declared equivalent and merged.

1.7.3 Table Filling (Myhill-Nerode) Method

The table filling method builds a triangular table of all state pairs, marks pairs (final, non-final) as distinguishable immediately, then repeatedly scans remaining pairs to mark more pairs distinguishable if any input symbol sends them to an already distinguishable pair.

This process repeats until one full pass makes no new marks, at which point every unmarked pair of states is guaranteed equivalent and can be merged into a single state.

The final minimized DFA is built by combining each group of mutually equivalent states into one representative state, keeping transitions consistent.

- Step 1: remove unreachable states from the original DFA.
- Step 2: mark every pair (final state, non-final state) as distinguishable.
- Step 3: for each unmarked pair (p, q), check every symbol a; if $(\delta(p,a), \delta(q,a))$ is already marked, mark (p, q) too.
- Step 4: repeat step 3 until no new pair is marked in a full pass.
- Step 5: merge every unmarked (still equivalent) pair into a single state to build the minimized DFA.

1.7.4 Minimization Algorithm and Complexity

The table filling method is simple to hand-trace for small DFAs, and it directly matches how SPPU expects the numerical to be solved step by step in the answer sheet.

A faster algorithm, Hopcroft's algorithm, achieves the same result in $O(n \log n)$ time using partition refinement, but the table filling method is preferred for manual exam solutions because it is easier to show as working.

Regardless of the algorithm used, the guarantee is the same: the resulting DFA is the unique smallest equivalent machine for the given language.

- Table filling method complexity: checks roughly $n^2/2$ pairs, each re-examined up to n times in the worst case.
- Hopcroft's algorithm complexity: $O(n \log n)$, preferred for large scale automated tools, not usually asked to derive in SPPU exams.

```
function MINIMIZE_DFA(D):
    remove all unreachable states from D
    mark(p,q) = TRUE for every pair where exactly one of p,q in F
    repeat:
        changed = false
        for each unmarked pair (p,q):
            for each symbol a in Sigma:
                if mark(delta(p,a), delta(q,a)) == TRUE:
                    mark(p,q) = TRUE
                    changed = true
    until changed == false
    merge all unmarked pairs into single states
    return new minimized DFA
```

Time Complexity: $O(n^2)$ per pass in simple table filling, $O(n \log n)$ using Hopcroft's algorithm **Space**

Complexity: $O(n^2)$ to store the state pair table

Worked Example 1

Given: A DFA has states $\{A,B,C,D,E\}$, $\Sigma=\{0,1\}$, A is start, $F=\{C\}$, with $A-0 \rightarrow B$, $A-1 \rightarrow C$, $B-0 \rightarrow A$, $B-1 \rightarrow D$, $C-0 \rightarrow E$, $C-1 \rightarrow C$, $D-0 \rightarrow E$, $D-1 \rightarrow C$, $E-0 \rightarrow E$, $E-1 \rightarrow E$.

Find: The minimized DFA using the table filling method.

Solution:

Step 1: Mark (C, X) as distinguishable for every non-final X, since C is the only final state: pairs (A,C),(B,C),(D,C),(E,C) are all marked.

Step 2: Check remaining pairs using transitions: (A,B) on 0 goes to (B,A) unmarked, on 1 goes to (C,D) which is marked, so mark (A,B) too.

Step 3: Check (A,D): on 0 goes to (B,E) unmarked so far, on 1 goes to (C,C) which is same state not distinguishable by this symbol; continue checking (B,E), (B,D), (D,E), (A,E) similarly using the marked table.

Step 4: After completing all rounds, unmarked pairs turn out to be (A,D) and (B,E) and (D,E) forming groups; merge D with E, keep A, B, C as they are distinguishable from each other and from the merged group.

Ans: Minimized DFA has 4 states: {A, B, C, merged-DE}, with the same accepted language as the original 5-state DFA

Advantages:

- Produces the unique smallest DFA for a given regular language, saving memory in real implementations.
- Speeds up execution slightly by reducing the number of states the machine must track internally.
- Table filling method is simple, systematic, and easy to verify by hand for exam purposes.
- Provides a standard way to check if two DFAs are equivalent, by minimizing both and comparing structure.
- Removing unreachable states first avoids wasted effort during the equivalence checking phase.

Disadvantages:

- Table filling method can be slow for very large DFAs, roughly $O(n^2)$ in the number of states.
- Manual tracing is tedious and error-prone for DFAs with many states and a large alphabet.
- Minimization only works correctly on a complete, fully reachable DFA — hidden unreachable states must be removed first or the result is wrong.

Applications:

- Compiler lexical analysers use minimized DFAs to keep the token recognizer as small and fast as possible.
- Digital circuit synthesis tools minimize state machines before generating hardware, reducing the number of flip-flops needed.
- Regular expression libraries minimize the compiled automaton to reduce memory footprint in production software.
- Network protocol validators minimize their state machines to simplify formal verification and reduce testing effort.

KEY POINTS — Minimization of DFAs

Always remove unreachable states first — minimizing before this step gives a wrong or incomplete answer.

A final state and a non-final state can never be equivalent — this is the starting mark in every table filling solution.

The minimized DFA is unique for a given regular language, up to renaming of states.

Common exam trap: stopping the table filling process too early — repeat rounds until a full pass makes no new marks.

Table filling is $O(n^2)$ roughly, while Hopcroft's algorithm achieves $O(n \log n)$ — know both names even if only table filling is solved by hand.

SPPU EXPECTED QUESTIONS — Minimization of DFAs

- Q1. Explain the concept of equivalent states in a DFA with the Myhill-Nerode theorem. (5 marks)
- Q2. Explain the table filling method for DFA minimization with its algorithm steps. (6 marks)
- Q3. Minimize the given DFA using the table filling method, showing the complete table. (8 marks)
- Q4. Why must unreachable states be removed before minimization? Explain with an example. (5 marks)
- Q5. Compare table filling method and Hopcroft's algorithm for DFA minimization. (6 marks)

1.8 Moore and Mealy Machines — Definition, Construction and Inter-Conversion

Definition:

Moore and Mealy machines are finite automata with output: a Moore machine produces an output associated with each state, while a Mealy machine produces an output associated with each transition, and both are used to model systems that generate a response, not just accept or reject a string.

Introduction / Need:

DFA and NFA only answer accept or reject, but many real systems, such as a sequence detector circuit or a vending machine controller, must produce an actual output signal as they run. Moore and Mealy machines extend finite automata to handle exactly this need.

This is usually the last and most numerically rich topic of Unit 1 in SPPU papers, often carrying an 8 mark question on construction or inter-conversion, so worked practice here is essential.

1.8.1 Moore Machine — Definition and Construction

A Moore machine is defined as a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, where Δ is the output alphabet and $\lambda: Q \rightarrow \Delta$ is the output function that assigns one fixed output symbol to every state.

Because output depends only on the current state, a Moore machine produces one extra output symbol compared to the length of the input — including the output of the initial state before any symbol is read.

Construction begins by deciding what condition each state represents, then assigning the correct output symbol to that state using λ , in addition to the usual δ transitions.

- Δ : finite output alphabet, e.g. $\Delta = \{0, 1\}$.
- $\lambda: Q \rightarrow \Delta$, output function, one output per state (not per transition).
- Output string length = input length + 1, since the start state also emits an output before reading anything.

Formula Name: Moore Machine Definition

Expression: $M = (Q, \Sigma, \Delta, \delta, \lambda, q_0), \lambda: Q \rightarrow \Delta$

Where:

Q = finite set of states

Σ = input alphabet

Δ = output alphabet

$\delta: Q \times \Sigma \rightarrow Q$, transition function

$\lambda: Q \rightarrow \Delta$, output function tied to states

q_0 = initial state

Condition: Output is generated once per state visited, so output length = |input| + 1.

1.8.2 Mealy Machine — Definition and Construction

A Mealy machine is also a 6-tuple $(Q, \Sigma, \Delta, \delta, \lambda, q_0)$, but here $\lambda: Q \times \Sigma \rightarrow \Delta$ assigns an output to every transition, based on both the current state and the input symbol read.

Since output is tied to transitions, a Mealy machine produces exactly one output symbol per input symbol, so the output length exactly equals the input length, unlike Moore machine.

Construction requires labelling each transition arrow with both the input symbol and the corresponding output symbol, typically written as 'input / output' on the arrow.

- $\lambda: Q \times \Sigma \rightarrow \Delta$, output function tied to transitions (state and symbol pair).
- Output string length = input length exactly, since no output is produced before the first symbol is read.
- Mealy machines often need fewer states than an equivalent Moore machine for the same output behaviour.

1.8.3 Difference between Moore and Mealy Machine

The fundamental difference is where the output function is attached: to states in Moore machine, and to transitions in Mealy machine, which changes both output length and machine size.

Mealy machines usually need fewer states than Moore machines for the same behaviour, because output can change based on the input symbol without needing a completely new state.

Moore machine output changes only when the state changes, making its output more stable and easier to predict, which is preferred in many hardware synchronous circuits.

- Output timing: Moore output is available as soon as a state is entered; Mealy output appears only while a transition is taken.
- Output length: Moore gives $|w|+1$ outputs; Mealy gives exactly $|w|$ outputs for input w .
- State count: Mealy machines are often more compact for the same function.

Sr. No.	Feature	Moore Machine	Mealy Machine
1	Output depends on	Current state only	Current state and input symbol
2	Output function	$\lambda: Q \rightarrow \Delta$	$\lambda: Q \times \Sigma \rightarrow \Delta$
3	Output string length	Input length + 1	Equal to input length

Sr. No.	Feature	Moore Machine	Mealy Machine
4	Number of states (typical)	Usually more	Usually fewer
5	Output stability	Changes only on state change	Can change with every input symbol

1.8.4 Conversion of Mealy Machine to Moore Machine

To convert Mealy to Moore, every state of the Mealy machine may need to be split into several new states, one for each distinct output value that could arrive at that state on different incoming transitions.

Each new state (state, output) pair carries that fixed output as its Moore output λ , and transitions are redirected to the correct split state based on incoming output value.

This conversion can increase the number of states, but the resulting Moore machine always produces the same input-output correspondence as the original Mealy machine, aside from one extra leading output symbol.

- Step 1: identify all distinct outputs associated with each Mealy state across its incoming transitions.
- Step 2: create one new Moore state per (state, output) pair found.
- Step 3: redirect all outgoing transitions of the original state to every new split copy identically.
- Step 4: assign the initial state an arbitrary but conventionally chosen output, often matching one incoming case.

1.8.5 Conversion of Moore Machine to Mealy Machine

Converting Moore to Mealy is simpler: every transition into a state q in the Mealy machine is labelled with the fixed output $\lambda(q)$ that the Moore machine had assigned to q .

No new states are needed for this direction, because Mealy machines can freely assign different outputs to different transitions leading into the same state.

This conversion always removes exactly one output symbol compared to the original Moore output, since Mealy has no output for the starting state before any symbol is read.

- For every transition $\delta(p, a) = q$ in the Moore machine, set the new Mealy output $\lambda'(p, a) = \lambda(q)$, the Moore output of the destination state.
- States and transition function δ remain unchanged; only the output labelling method changes.
- The very first output symbol of the Moore machine (for q_0 before reading anything) is simply dropped, since Mealy has no equivalent.

Worked Example 1

Given: A Moore machine has states q_0 (output 0), q_1 (output 1), with $\delta(q_0, a) = q_1$, $\delta(q_0, b) = q_0$, $\delta(q_1, a) = q_1$, $\delta(q_1, b) = q_0$, q_0 is start state.

Find: The equivalent Mealy machine and the output for input string 'aab'.

Solution:

Step 1: Apply rule: label every transition into a state q with output $\lambda(q)$ from the Moore machine.

Step 2: Transition $q_0 \text{-a-} \rightarrow q_1$ gets output $\lambda(q_1)=1$; transition $q_0 \text{-b-} \rightarrow q_0$ gets output $\lambda(q_0)=0$; $q_1 \text{-a-} \rightarrow q_1$ gets output 1; $q_1 \text{-b-} \rightarrow q_0$ gets output 0.

Step 3: Trace Moore output for 'aab' first: start output $\lambda(q_0)=0$, then $q_0 \text{-a-} \rightarrow q_1$ output 1, $q_1 \text{-a-} \rightarrow q_1$ output 1, $q_1 \text{-b-} \rightarrow q_0$ output 0, giving Moore output sequence 0,1,1,0 (length 4).

Step 4: Trace Mealy output for the same string using new labels: $q_0 \text{-a-} \rightarrow q_1$ gives 1, $q_1 \text{-a-} \rightarrow q_1$ gives 1, $q_1 \text{-b-} \rightarrow q_0$ gives 0, giving Mealy output sequence 1,1,0 (length 3, one less than Moore).

Ans: Mealy output for 'aab' = 110 (length 3); Moore output for 'aab' = 0110 (length 4, includes initial state output)

Worked Example 2

Given: Design a Mealy machine over $\Sigma=\{0,1\}$ that outputs 1 whenever the last two input symbols read are '11', and 0 otherwise.

Find: The states, transitions with output labels for this sequence detector.

Solution:

Step 1: Identify memory needed: whether the previous symbol was 1 or not, so 2 states suffice: q_A = last symbol was not 1 (start), q_B = last symbol was 1.

Step 2: From q_A on 0: stay q_A , output 0 (no 11 seen); from q_A on 1: go to q_B , output 0 (only one 1 seen so far, not enough).

Step 3: From q_B on 1: stay q_B , output 1 (two consecutive 1's detected); from q_B on 0: go to q_A , output 0 (sequence broken).

Step 4: Trace input '1101': $q_A \text{-1/0-} \rightarrow q_B \text{-1/1-} \rightarrow q_B \text{-0/0-} \rightarrow q_A \text{-1/0-} \rightarrow q_B$, giving output sequence 0,1,0,0.

Ans: Mealy machine has 2 states $\{q_A, q_B\}$; output for input '1101' = 0100

Advantages:

- Model real sequential systems that must produce output, not just accept or reject a string.
- Moore machine gives more stable output since it changes only on state transitions, useful in synchronous hardware design.
- Mealy machine usually needs fewer states, giving a smaller, more efficient circuit implementation.
- Directly maps onto sequential digital circuits built from flip-flops and combinational logic.
- Inter-conversion allows a designer to pick whichever model is more convenient and still implement the same required behaviour.

Disadvantages:

- Mealy machine output can glitch or change asynchronously with input, which sometimes complicates hardware timing analysis.
- Moore machine may require more states than Mealy for the same function, increasing hardware or memory cost.
- Converting Mealy to Moore can significantly increase the number of states due to output based state splitting.
- Manual construction and tracing of output sequences is more error-prone than plain DFA/NFA due to the extra output labels.

Applications:

- Sequence detector circuits in digital electronics, such as detecting a specific bit pattern in a serial data stream.
- Traffic light controllers, modelled as Moore machines where each state directly represents one light combination as output.
- Vending machines and elevator controllers, which must output actions (dispense item, open door) tied to specific states or transitions.
- Communication protocol controllers that must emit acknowledgement signals based on the sequence of received messages.
- Digital counters and simple ALU control units implemented as Mealy machines in processor design.

KEY POINTS — Moore and Mealy Machines — Definition, Construction and Inter-Conversion

Moore output is tied to STATES ($\lambda: Q \rightarrow \Delta$), Mealy output is tied to TRANSITIONS ($\lambda: Q \times \Sigma \rightarrow \Delta$) — this single fact answers most exam questions on this topic.

Moore output length = input length + 1; Mealy output length = input length exactly — a very common numerical trap.

Converting Moore to Mealy never adds states; converting Mealy to Moore can add states due to output-based splitting.

When converting Mealy to Moore, remember to assign an output to the new start state even though no transition created it.

SPPU almost always asks either a construction question or an inter-conversion numerical from this topic — practise both directions.

SPPU EXPECTED QUESTIONS — Moore and Mealy Machines — Definition, Construction and Inter-Conversion

Q1. Define Moore machine and Mealy machine with their formal 6-tuple notations. (5 marks)

Q2. Differentiate between Moore machine and Mealy machine with respect to output and states. (6 marks)

Q3. Convert the given Mealy machine into an equivalent Moore machine, showing all steps. (8 marks)

Q4. Design a Mealy machine over $\{0,1\}$ that outputs 1 when the last two symbols are '11'. (6 marks)

Q5. Convert the given Moore machine into an equivalent Mealy machine and verify output for a given input string. (8 marks)

Unit Summary

UNIT I SUMMARY

1.1 Sets, Symbols and Strings — a language is a set of strings built from symbols of a finite alphabet Σ ; master Σ^* , Σ^+ and ϵ .

1.2 Formal Language and Operations — union, concatenation, Kleene star (L^*) and Kleene plus (L^+) build complex languages from simple ones; L^* always includes ϵ .

1.3 Concept of Basic Machine — every automaton shares states, an alphabet and a transition rule; split is FA without output vs FA with output.

1.4 DFA — 5-tuple $(Q, \Sigma, \delta, q_0, F)$, δ total and single valued, $L(M) = \{w \mid \delta^*(q_0, w) \in F\}$, simulation is $O(n)$ time.

1.5 NFA — $\delta: Q \times \Sigma \rightarrow 2^Q$, acceptance is existential (at least one path to a final state); NFA and DFA are equally powerful.

1.6 Epsilon-NFA and Conversions — ϵ -closure(q) always includes q ; subset construction converts NFA (or NFA- ϵ) to an equivalent DFA, worst case $2^{|Q|}$ states.

1.7 Minimization of DFA — table filling method marks (final, non-final) pairs first, then propagates distinguishability; result is the unique smallest DFA.

1.8 Moore and Mealy Machines — Moore output tied to states (output length = input+1), Mealy output tied to transitions (output length = input); inter-conversion is a frequent numerical.

Key formula to remember: $L^* = \bigcup_{n=0}^{\infty} L^n$, and $L^+ = LL^*$.

Most common SPPU exam question across this unit: construct or convert an automaton (DFA/NFA/Moore/Mealy) step by step, or minimize a given DFA using table filling.